Estructuras de control y bucles en JAVA - José Ángel Alejo Sillero

- Estructuras de control y bucles en JAVA - José Ángel Alejo Sillero
  - ALTERNATIVAS
    - Simples - IF
    - Dobles - IF, ELSE
    - MÚLTIPLES
      - Switch
        - Switch "Normal"
        - Switch "Moderno"
      - IF-ELSE ANIDADOS
  - BUCLES → While | Do While | For
    - While
    - Do-While
    - For
  - Tabla comparativa de estructuras de control y bucles en Java
    - Consideraciones:

Las estructuras de control en Java se dividen en Alternativas y Bucles.

En las **Alternativas** tenemos: Simples (IF), los Dobles (IF, ELSE) y Múltiples (switch normal, switch moderno y if-else anidado).

En los **Bucles** tenemos: While, Do_While y For.

ALTERNATIVAS

En las **Alternativas** tenemos: Simples (IF), los Dobles (IF, ELSE) y Múltiples (switch normal, switch moderno y if-else anidado).

Es muy importante prestar atención a mantener la estructura correcta con el uso de llaves `{}` y comas `,`.

Simples - IF

Se utilizan para establecer qué pasará si se cumple una condición (IF - si)

```
int n = 3;
int n1 = 9;
if (n1 > n) {
    System.out.println(n1 + " Es mayor que" + n);       // En este ejemplo, el símbolo
+ no suma. Lo utilizamos para mostrar el valor de las variables `n1` y `n` que están
declaradas
}
```

Dobles - IF, ELSE

```
int n = 3;
int n1 = 9;
if (n1 > n) {
    System.out.println(n1 + " Es mayor que" + n);       // En este ejemplo, el símbolo
+ no suma. Lo utilizamos para mostrar el valor de las variables `n1` y `n` que están
declaradas
}
else {
    System.out.println(n1 + "No es mayor que" + n);     // Con "ELSE" establecemos lo
que pasa cuando no se cumple "IF"
}
```

MÚLTIPLES

Una estructura de control será Múltiple cuando usamos Switch

Switch

Los switch se utilizan con valores exactos. Es muy conveniente ya que directamente se va a la condición correspondiente al valor que le corresponda.

Un switch solo puede utilizar valores exactos y constantes (no variables).

Switch "Normal"

```
int dia = 2;
switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;                              // Usamos break para que no siga
ejecutando los siguientes case.
    case 2:
        System.out.println("Martes");       // En caso de no utilizar break, seguiría
ejecutando los siguientes case hasta encontrar un break o finalizar el switch.
        break;                              // Esto provoca una caida de flujo y puede
ser útil en algunos casos.
    case 3:
        System.out.println("Miercoles");
        break;
    case 4:
        System.out.println("Jueves");
        break;
    case 5:
        System.out.println("Viernes");
        break;
    case 6, 7:                              // Podemos agrupar varios case que hagan
lo mismo.
        System.out.println("Sabado");
        break;
    default:                                // El default es opcional. Se ejecuta si
no se cumple ningún case.
        System.out.println("Valor no valido");
}
```

Switch "Moderno"

Desde Java 14 podemos usar switch de una forma más simplificada, usando flechas "->" en lugar de ":" y "break". En este caso no es necesario usar "break", ya que la flecha indica el final del case.

Este tipo de Switch está muy bien porque puedes hacer que te devuelva un valor y hacer que se guarde en una variable.

```
int dia = 2;
switch (dia) {
    case 1-> System.out.println("Lunes");
    case 2-> System.out.println("Martes");
    case 3-> System.out.println("Miercoles");
    case 6, 7-> System.out.println("Es fin de semana");
    default-> System.out.println("Valor no valido");
}
```

Este tipo de Switch está muy bien podemos hacer que el Switch devuelva un valor y lo guarde en una variable

```
String diaSemana;
diaSemana = switch (dia) {
    case 1-> "Lunes";
    case 2-> "Martes";
    case 3-> "Miercoles";
    case 6, 7-> "Fin de semana";
    default-> "Valor no valido";
};
System.out.println(diaSemana);
```

IF-ELSE ANIDADOS

Los IF-Else anidados son aquellos que contienen dentro de su bloque otro IF-Else. En IF-Else anidado, cuando se cumple una condición, se ignoran las demás condiciones.

```
float nota = 7.5f;

if (nota >= 0 && nota < 5){
    System.out.println("Suspenso");
}
else if (nota<6) {  // No es necesario indicar que debe "nota>= 0 && nota<6". Se asume que será = o mayor a 0 al no cumplirse la condición IF anterior
    System.out.println("Suficiente");
}
else if (nota<7) {
    System.out.println("Bien");
}
else if (nota<9) {
    System.out.println("Notable");
}
else if (nota<=10) {
    System.out.println("Sobresaliente");
}
else if (nota>10 || nota<0) {           // Agregamos "|| nota<0" para controlar notas no válidas.
    System.out.println("Nota no válida");
}
```

BUCLES → While | Do While | For

Un bucle es un trozo de código que se repite mientras una condición se cumpla(sea verdadera).

While

Se utiliza cuando **no se sabe cuántas veces se va a repetir el bloque de código.**

Se lee: mientras (while) "condición", haz(do) "bloque de código" contenido.

Ejemplo de estructura básica de while:

```
while (condición) {
    // bloque de código
}
```

Ejemplo de While:

```
int num2 = 0;
int suma = 0;
while (suma <= 100) {
    System.out.print("Introduce un número para sumar: ");
    num2 = scanner.nextInt();   // Leemos el número introducido por el usuario
    suma = suma + num2;         // Se puede indicar como suma += num2;
}
scanner.close();
```

Do-While

Parecido al While, pero se ejecuta **primero el Do y luego se comprueba la condición en el While.**

Esto significa que el bloque de código se ejecutará al menos una vez aunque la condición no se cumpla.

Se lee: haz (do) "bloque de código", mientras (while) "condición".

Lo **utilizamos cuando queremos que el bloque de código se ejecute al menos una vez.**

Estructura básica de do-while:

```
do {
    // bloque de código
} while (condición);
```

Ejemplo de Do-While:

```
Scanner scanner = new Scanner(System.in);
// Bucle Do-While
int num3 = 0;
int suma = 0;
do {
    System.out.print("Introduce un número mayor que 0: ");
    num3 = scanner.nextInt();       // Leemos el número introducido por el usuario
    suma = suma + num3;             // Se puede indicar como suma += num3;
} while (num3 <= 0);

scanner.close();        // Cerramos el scanner
```

For

El bucle For se utiliza cuando **sabemos cuántas veces se va a repetir el bloque de código.**

Se lee: para (for) "inicialización; condición(hasta); incremento/decremento", haz "bloque de código".

Estructura básica de For:

```
for (inicialización; condición; incremento/decremento) {
    // bloque de código
}
```

Ejemplo de For:

```
for (int num5=8 ; num5 <= 5 ; num5++) {     // Para num5 empezando en 8, mientras num5
sea menor o igual a 5, incrementa num5 en 1.
    System.out.println("Número: " + num5);
}
```

Ejemplo para múltiplos de 3, del 3 al 15

```
for (int num6=3 ; num6 <= 15 ; num6+=3) {         // Para num6 empezando en 3,
mientras num6 sea menor o igual a 15, incrementa num6 en 3.
    System.out.println("Número: " + num6);
}
```

Para un enunciado: Pide un número por pantall y muestra su tabla de multiplicar del 1 al 10, sería:

```
Scanner scanner2 = new Scanner(System.in);
System.out.print("Introduce un número para ver su tabla de multiplicar: ");

int numTabla = scanner2.nextInt();
for (int multiplicador=1 ; multiplicador <=10 ; multiplicador++) {

    int resultado = numTabla * multiplicador;
    System.out.println(numTabla +  "x" +  multiplicador +  "=" +  resultado);
}
scanner2.close();
```

Tabla comparativa de estructuras de control y bucles en Java

Estructura	Cuándo se utiliza	Sintaxis Básica	Se lee
If	Evaluar una condición y ejecutar código si es verdadera	if (condición) { // código }	Si (condicion) se cumple, ejecutar código.
If-Else	Evaluar una condición y ejecutar código si es verdadera o falsa.	if (condición) { // código } else { // código }	Si (condicion) se cumple, ejecutar código. Si no, ejecutar otro código.
Switch	Seleccionar entre múltiples opciones basadas en el valor de una variable	switch (variable) { case valor1: // código; break; ... }	Si (variable) es igual a valor, ejecutar código asociado.
BUCLES	**Repetir un bloque de código mientras se cumpla una condición**.		
While	Cuando **no sabemos cuántas veces** se debe repetir un bloque de código.	while (condición) { // código }	Mientras (condicion) se cumpla, ejecutar código.
Do-While	Como el While, pero el **bloque se ejecutará al menos una vez**.	do { // código } while (condición);	Hacer código, mientras (condicion) se cumpla.
For	Cuando **sabemos cuántas veces se repetirá el bloque y usamos un contador**.	for (inicialización; condición; incremento) { // código }	Para (inicialización; condición; incremento), ejecutar código.

Consideraciones:

- En los **bucles**, es importante **asegurarse de que la condición eventualmente se vuelva falsa para evitar bucles infinitos**.
- En los **bucles la variable de control (contador) debe ser declarada antes del bucle** si se va a utilizar después del mismo.
- Si queremos **imprimir un valor que no se calcula hasta que termine el bucle**, hacemos la **impresión fuera del bucle**.
- Si queremos imprimir un valor que se **calcula en cada iteración del bucle, hacemos la impresión dentro del bucle**.
- Los **operadores de incremento y decremento** son:
  - `++` es el operador de incremento que aumenta el valor de una variable en 1. Puede usarse como prefijo (`++i`) o sufijo (`i++`).
  - `--` es el operador de decremento que disminuye el valor de una variable en 1. Puede usarse como prefijo (`--i`) o sufijo (`i--`).
- Operadores de **asignación compuesta**:
  - `+=` y `-=` Permiten incrementar o decrementar una variable en una cantidad específica. Por ejemplo, `i += 2` es equivalente a `i = i + 2`.
  - `*=` y `/=` Permiten multiplicar o dividir una variable en una cantidad específica. Por ejemplo, `i *= 3` es equivalente a `i = i * 3`.